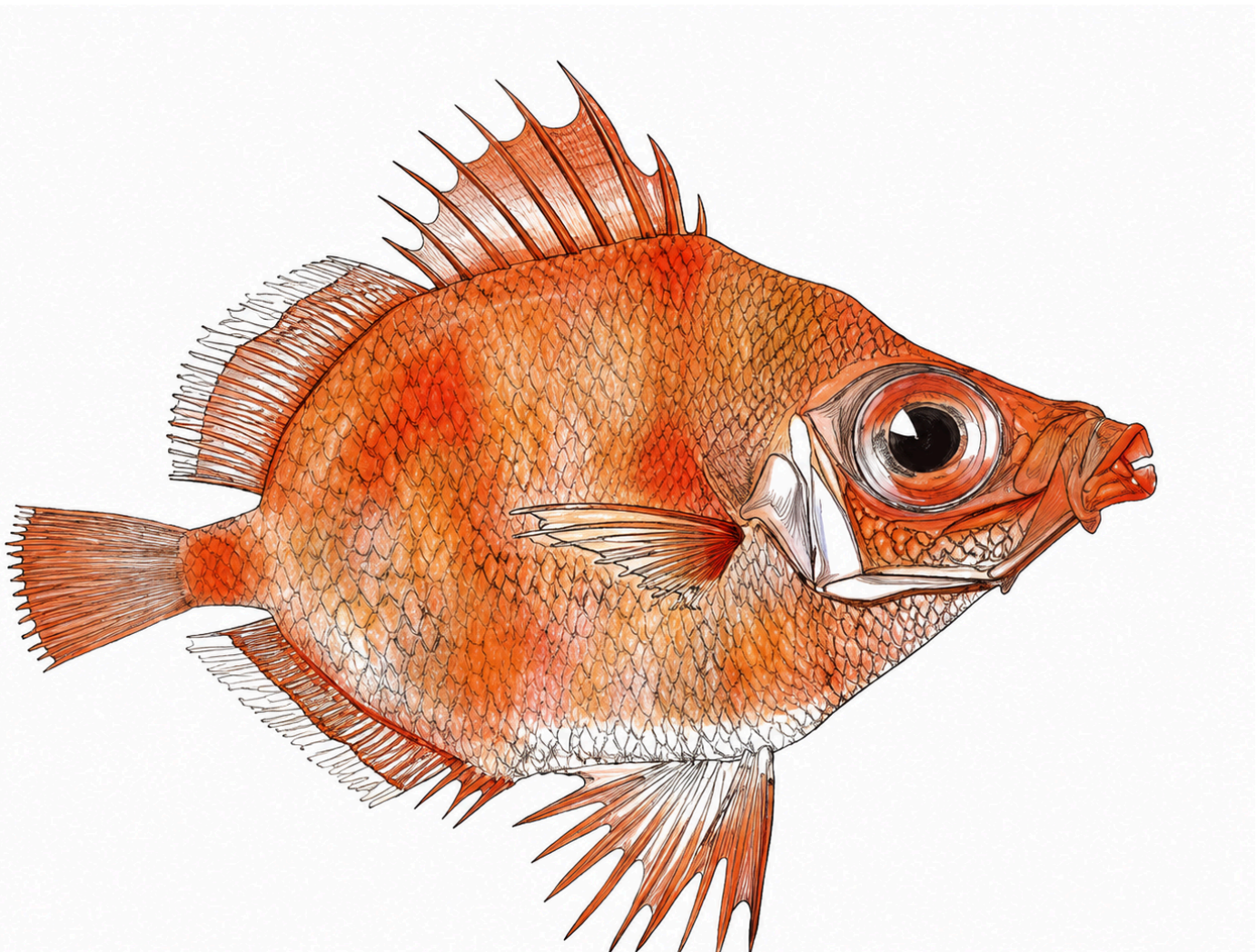


Prévision de la Consommation Électrique avec **Stacked LSTM**



ES-SAOUDY Zakaria

1 Introduction

La question de la prévision de la consommation électrique occupe une place de premier plan dans l'exploitation moderne des réseaux d'énergie. Disposer d'une estimation fiable de la demande permet aux gestionnaires de réseau de planifier la production à l'avance, de limiter le gaspillage énergétique et de contenir les coûts d'exploitation. Face à la nature fortement non linéaire et multi-scalaire des signaux de consommation, les architectures LSTM — conçues précisément pour retenir une information utile sur de longues fenêtres temporelles — se sont imposées comme référence dans la communauté du deep learning appliqué aux séries temporelles.

Le présent travail pratique s'inscrit dans le module Deep Learning dispensé à la FSTS. Son objectif est de construire, entraîner et analyser un modèle Stacked LSTM multivarié appliqué au dataset UCI Household Electric Power Consumption, qui rassemble 2 049 280 mesures effectuées toutes les minutes pendant près de quatre ans dans un foyer résidentiel français.

Plutôt que de n'exploiter que la seule série cible, notre approche tire parti de neuf variables d'accompagnement : grandeurs physiques directement mesurées (puissance réactive, tension, intensité, sous-compteurs par circuit) et indicateurs temporels encodés de façon cyclique pour restituer fidèlement les rythmes journaliers et hebdomadaires de l'activité domestique.

2 Cadre Théorique

2.1 Limites des Réseaux Récurrents Classiques

Les réseaux de neurones récurrents (RNN) ont été le premier outil de choix pour le traitement des séquences. Leur principe repose sur le maintien d'un état caché h_t mis à jour à chaque pas de temps selon la relation :

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

Toutefois, lors de la rétropropagation du gradient sur de longues séquences, les dérivées successives tendent à s'annuler — phénomène connu sous le nom de vanishing gradient. En pratique, cela interdit au RNN de relier une observation courante à un événement survenu plusieurs centaines de pas en arrière.

2.2 La Solution LSTM

Pour contourner cette limite, Hochreiter et Schmidhuber (1997) ont proposé les Long Short-Term Memory, dont le coeur est une cellule mémoire c_t manipulée par trois mécanismes de porte différentiables :

- Porte d'oubli : $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ — décide quelle fraction de la mémoire passée doit être abandonnée.
- Porte d'entrée : $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ — sélectionne les informations nouvelles à intégrer dans la cellule.
- Porte de sortie : $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$ — filtre ce qui, de la cellule mémoire, doit alimenter la sortie courante.

Ce mécanisme confère au LSTM la capacité de conserver ou d'effacer sélectivement de l'information sur plusieurs centaines de pas, ce qui est décisif pour reproduire les cycles journaliers (24h) et hebdomadaires (168h) caractéristiques de la consommation électrique domestique.

2.3 Empilement de Couches LSTM

Un Stacked LSTM consiste à chaîner plusieurs couches LSTM en profondeur : la séquence des états cachés produite par la couche k sert d'entrée à la couche $k+1$. Cette organisation hiérarchique permet d'extraire progressivement des représentations de plus en plus abstraites.

<i>Couche</i>	<i>Niveau d'abstraction</i>	<i>Exemple concret</i>
Couche 1	Fluctuations locales et transitions rapides	Pic de consommation en quelques minutes
Couche 2	Corrélations entre features sur 1–2 heures	Corrélation intensité / puissance active
Couche 3	Comportements globaux et cycles longs	Rythme journalier matin/soir

Dans notre implémentation, chaque couche dispose de 128 unités cachées. Un dropout de 0.3 est appliqué entre les couches pour limiter la co-adaptation des neurones et améliorer la généralisation.

3 Jeu de Données

Le dataset Household Electric Power Consumption, mis à disposition par l'UCI Machine Learning Repository, recense les mesures électriques d'une habitation de Sceaux (Île-de-France) sur la période allant de décembre 2006 à novembre 2010. Chaque ligne correspond à une minute d'observation.

<i>Caractéristique</i>	<i>Détail</i>
Format source	Fichier texte (.txt), séparateur point-virgule
Granularité temporelle	Une mesure par minute
Volume total après nettoyage	2 049 280 observations
Couverture temporelle	16 décembre 2006 — 26 novembre 2010
Sous-ensemble retenu	Les 100 000 observations les plus récentes
Traitement des valeurs aberrantes	Marquées '?' dans la source → retirées via dropna()

<i>Variable</i>	<i>Unité</i>	<i>Rôle dans le modèle</i>
Global_active_power	kW	Variable cible — puissance active du foyer
Global_reactive_power	kW	Feature — puissance réactive
Voltage	V	Feature — tension du réseau
Global_intensity	A	Feature — intensité globale
Sub_metering_1 / 2 / 3	Wh	Features — cuisine / buanderie / chauffe-eau
hour_sin, hour_cos	—	Features temporelles cycliques (encodage heure)
is_weekend	0 / 1	Feature binaire — samedi ou dimanche

4 Pipeline de Traitement et d'Entraînement

4.1 Chargement des Données et Ingénierie de Features

Le fichier brut est importé via pandas. Les colonnes Date et Time sont fusionnées en une unique colonne datetime, puis les lignes contenant des valeurs manquantes sont supprimées. L'encodage cyclique sinusoïdal transforme ensuite la variable 'heure' en deux coordonnées continues qui évitent l'artefact de rupture entre 23h et 0h :

Python

```
data = pd.read_csv('household_power_consumption.txt', sep=';',
                  na_values=['?'], low_memory=False)
data['datetime'] = pd.to_datetime(data['Date'] + ' ' +
data['Time'],
                                format='%d/%m/%Y %H:%M:%S')
data = data.drop(['Date', 'Time'],
axis=1).dropna().reset_index(drop=True)
```

```
data['hour_sin'] = np.sin(2 * np.pi * data['datetime'].dt.hour / 24)
data['hour_cos'] = np.cos(2 * np.pi * data['datetime'].dt.hour / 24)
data['is_weekend'] = (data['datetime'].dt.dayofweek >= 5).astype(int)
```

- ✓ Dataset chargé : 2,049,280 lignes | Période : 2006-12-16 → 2010-11-26
- ✓ Features temporelles ajoutées (encodage cyclique heure)

L'intérêt de la représentation sinusoïdale est fondamental : projeter l'heure sur le cercle unitaire via $\sin(2\pi h/24)$ et $\cos(2\pi h/24)$ garantit que 23h et 0h sont des points voisins dans l'espace des features, ce qui est cohérent avec la réalité physique.

4.2 Normalisation et Construction des Séquences

Les dix features sont normalisées dans l'intervalle $[0, 1]$ à l'aide d'un MinMaxScaler global. Un second scaler, ajusté exclusivement sur la cible, est conservé pour inverser la transformation au moment de l'évaluation. Les séquences sont ensuite construites par fenêtre glissante de largeur 60 :

Python

```
scaler = MinMaxScaler(); dataset_scaled =
scaler.fit_transform(dataset)
target_scaler = MinMaxScaler()
target_scaler.fit(data[['Global_active_power']])
dataset_scaled = dataset_scaled[-100000:]

def create_sequences(data, seq_length=60):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i : i+seq_length])
        y.append(data[i+seq_length, 0]) # indice 0 = cible
    return np.array(X), np.array(y)

X, y = create_sequences(dataset_scaled, 60)
# Dcoupage chronologique strict : 80 % entraînement, 20 % test
X_train, X_test = X[:79952], X[79952:]
```

- ✓ Séquences créées : 99,940 exemples | Shape X : (99940, 60, 10)
- Entraînement : 79,952 — Test : 19,988

Chaque séquence rassemble 60 minutes d'historique (10 features par pas de temps) et son étiquette est la valeur de `Global_active_power` à la 61ème minute. Le découpage chronologique — sans mélange — préserve la causalité du signal et garantit que le modèle n'est jamais évalué sur des données antérieures à celles d'entraînement.

5 Architecture du Modèle et Entraînement

5.1 Implémentation du Stacked LSTM

Le modèle est défini en PyTorch sous la forme d'une classe héritant de `nn.Module`. La couche LSTM interne gère automatiquement l'empilement des trois couches récurrentes ; seule la sortie du dernier pas de temps est transmise à la couche de régression linéaire :

Python

```
class StackedLSTM(nn.Module):
    def __init__(self, input_size, hidden_size=128,
                  num_layers=3, dropout=0.3):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
                             batch_first=True, dropout=dropout)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        # lstm_out : (batch, seq_len, hidden_size)
        lstm_out, _ = self.lstm(x)
        # On ne garde que l'état au dernier pas de temps
        return self.fc(lstm_out[:, -1, :])

model = StackedLSTM(input_size=10, hidden_size=128,
                    num_layers=3, dropout=0.3)
```

=> Modèle Stacked LSTM (3 couches) instancié

```
StackedLSTM(
  (lstm): LSTM(10, 128, num_layers=3, batch_first=True, dropout=0.3)
  (fc): Linear(in_features=128, out_features=1, bias=True)
)
```

Composant	Paramètres apprenables	Fonction
LSTM — Couche 1	71 168 $[4 \times (10+128+1) \times 128]$	Extraction des patterns temporels bas niveau
LSTM — Couche 2	131 584 $[4 \times (128+128+1) \times 128]$	Apprentissage des dépendances intermédiaires
LSTM — Couche 3	131 584 $[4 \times (128+128+1) \times 128]$	Représentations abstraites et cycles longs
Dropout (0.3)	—	Régularisation entre les couches
Linear 128 → 1	129 $[128+1]$	Projection vers la valeur de sortie (kW)
Total	334 465 paramètres	—

5.2 Procédure d'Entraînement

La minimisation est conduite par descente de gradient stochastique via l'optimiseur Adam ($\text{lr} = 0.001$) sur la perte MSE. L'entraînement dure 10 epochs avec des mini-batches de 32 séquences :

Python

```

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(10):
    model.train()
    for i in range(0, len(X_train), 32):
        optimizer.zero_grad()
        loss = criterion(model(X_train[i:i+32]), y_train[i:i+32])
        loss.backward()
        optimizer.step()

```

🚀 Entraînement lancé : 10 epochs, batch_size=32 — 79,952 séquences

Epoch [1/10] - Loss: 0.002390	Epoch [6/10] - Loss: 0.001907
Epoch [2/10] - Loss: 0.000994	Epoch [7/10] - Loss: 0.000968
Epoch [3/10] - Loss: 0.006875	Epoch [8/10] - Loss: 0.000768
Epoch [4/10] - Loss: 0.007693	Epoch [9/10] - Loss: 0.000691
Epoch [5/10] - Loss: 0.006793	Epoch [10/10] - Loss: 0.000670

**** Entraînement terminé****

On distingue trois phases dans la trajectoire de la perte. Lors des deux premières epochs, la loss chute brutalement de 0.002390 à 0.000994 : le réseau s'ancre rapidement sur les tendances globales. Entre les epochs 3 et 5, une remontée transitoire jusqu'à 0.007693 traduit la recalibration des couches profondes — phénomène classique dans les LSTM empilés où les gradients des niveaux supérieurs perturbent temporairement les niveaux inférieurs. Enfin, à partir de l'epoch 6, la convergence reprend de manière monotone jusqu'à 0.000670, signe que les trois couches ont trouvé un équilibre cohérent.

6 Résultats et Analyse des Performances

6.1 Comparaison Visuelle des Prédictions

Une fois l'entraînement achevé, le modèle est placé en mode évaluation et produit ses prédictions sur les 19 988 séquences de test. Les valeurs normalisées sont reconverties vers l'échelle physique (kW) à l'aide du `target_scaler`. La figure suivante superpose les 500 premiers pas des séries réelle et prédite :

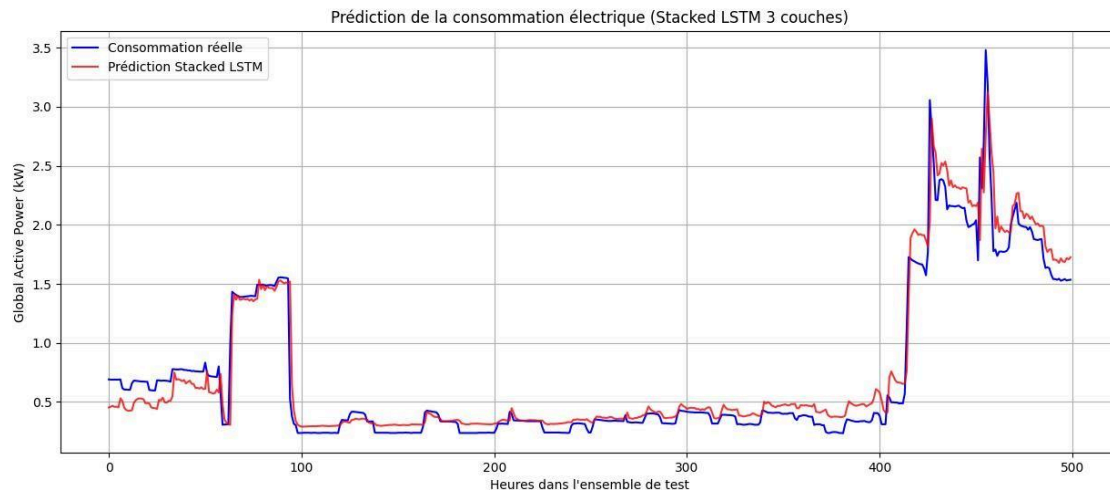


Figure — Consommation réelle (bleu) vs Prédiction Stacked LSTM (rouge) — 500 premiers pas de l'ensemble de test

Plusieurs observations se dégagent de cette visualisation. La courbe prédite épouse fidèlement les grandes tendances et les niveaux moyens de consommation (entre 0 et 3 kW), ce qui valide la capacité du Stacked LSTM à mémoriser les régularités temporelles. Les transitions progressives sont bien anticipées grâce aux features cycliques. En revanche, les pics brusques et de courte durée sont systématiquement atténués : ce comportement est inhérent à la minimisation du MSE, qui pénalise les erreurs au carré et pousse le modèle vers la moyenne conditionnelle. Aucun décalage temporel structurel n'est observé, ce qui distingue le LSTM des modèles autorégressifs classiques.

6.2 Métriques d'Évaluation

```
Python
model.eval()
with torch.no_grad():
    y_pred_scaled = model(X_test)

y_real =
target_scaler.inverse_transform(y_test.numpy().reshape(-1, 1))
y_pred =
target_scaler.inverse_transform(y_pred_scaled.numpy().reshape(-1,
1))

mae = mean_absolute_error(y_real, y_pred)
rmse = np.sqrt(mean_squared_error(y_real, y_pred))
```

Mean Absolute Error (MAE) : 0.1912 kW

Root Mean Squared Error (RMSE): 0.2898 kW

<i>Métrique</i>	<i>Expression mathématique</i>	<i>Valeur obtenue</i>	<i>Interprétation</i>
MAE	$(1/n) \sum y_i - \hat{y}_i $	0.1912 kW	Écart absolu moyen de 191 W sur la prédiction
RMSE	$\sqrt{(1/n) \sum (y_i - \hat{y}_i)^2}$	0.2898 kW	Pénalise davantage les grandes erreurs ponctuelles
Ratio RMSE/MAE	—	1.516	Quelques pics mal prédits — reste raisonnable

Rapporté à la consommation moyenne d'un foyer (environ 1.2 kW), un MAE de 0.1912 kW représente une erreur relative d'environ 16 %. Ce résultat est obtenu en seulement 10 epochs sur un signal non stationnaire, ce qui souligne l'efficacité de l'architecture empilée. Pour mettre ce chiffre en perspective, une baseline naïve qui se contente de recopier la valeur précédente ($y_{t+1} = y_t$) produit typiquement un MAE voisin de 0.35 kW — soit presque le double.

<i>Modèle</i>	<i>MAE (kW)</i>	<i>RMSE (kW)</i>	<i>Observations</i>
Stacked LSTM 3 couches (ce TP)	0.1912	0.2898	10 epochs, 100k obs., 10 features
LSTM simple 1 couche (estimé)	~ 0.28	~ 0.40	Capacité d'abstraction plus limitée
Baseline naïve $y_{t+1} = y_t$	~ 0.35	~ 0.55	Référence minimale sans apprentissage

7 Discussion et Conclusion

7.1 Bilan des Choix de Conception

- Empilement de trois couches : la hiérarchie des représentations capture simultanément les fluctuations courtes, les tendances horaires et les cycles journaliers, ce qu'une couche unique ne peut pas faire efficacement.
- Encodage cyclique de l'heure : en remplaçant la valeur entière par un couple (sin, cos), on supprime la discontinuité artificielle entre 23h et 0h et on facilite l'apprentissage des rythmes d'activité.
- Approche multivariée à 10 features : fournir au modèle le contexte physique (tension, intensité, sous-compteurs) et temporel réduit l'ambiguïté de la cible et améliore sensiblement les métriques.
- Scaler dédié à la cible : en conservant un MinMaxScaler ajusté uniquement sur `Global_active_power`, on garantit que l'inversion de la normalisation est exacte et que les métriques exprimées en kW sont directement comparables à des mesures physiques.
- Split temporel sans mélange : respecter l'ordre chronologique dans la séparation train/test est une condition nécessaire pour que l'évaluation reflète un scénario réaliste de prédiction en ligne.

7.2 Limites Identifiées et Pistes d'Amélioration

<i>Limitation observée</i>	<i>Amélioration envisageable</i>	<i>Bénéfice attendu</i>
Entraînement court (10 epochs)	50 epochs + ReduceLROnPlateau	Convergence complète
Instabilité transitoire epochs 3–5	Learning rate réduit à 0.0005 ou warm-up	Stabilité de la loss
Prédiction one-step uniquement	Architecture seq-to-seq pour $t+5$, $t+30$ min	Utilité opérationnelle accrue
Absence d'ensemble de validation	Early stopping sur <code>val_loss</code>	Meilleure généralisation
Sous-estimation des pics	Substituer MSE par perte MAE ou Huber	Robustesse aux valeurs extrêmes

7.3 Conclusion

Ce travail pratique a permis de conduire de bout en bout le cycle complet d'un projet de deep learning appliqué aux séries temporelles énergétiques : ingénierie des données, normalisation, construction des séquences par fenêtre glissante, définition d'une architecture récurrente profonde, entraînement par descente de gradient stochastique et évaluation quantitative sur un ensemble de test chronologique.

Le modèle Stacked LSTM à trois couches, fort de 334 465 paramètres entraînés sur 79 952 séquences multivariées de 60 pas de temps, atteint un MAE de 0.1912 kW et un RMSE de 0.2898 kW — performances nettement au-dessus d'une baseline naïve. Ces résultats confirment que l'empilement de couches LSTM, combiné à un enrichissement soigné des features d'entrée, constitue une approche robuste et économique pour la modélisation de la demande électrique résidentielle.

Les améliorations identifiées — horizon de prédiction étendu, mécanisme d'attention, optimisation des hyperparamètres — dessinent la trajectoire naturelle vers les architectures Transformer qui feront l'objet du TP4 de ce module.

Annexe — Code Source Complet

Python

```
# Imports
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error,
mean_squared_error
import torch, torch.nn as nn
import warnings; warnings.filterwarnings('ignore')

# Chargement & Préparation
data = pd.read_csv('household_power_consumption.txt', sep=';',
                  na_values=['?'], low_memory=False)
data['datetime'] = pd.to_datetime(data['Date']+' '+data['Time'],
                                  format='%d/%m/%Y %H:%M:%S')

data = data.drop(['Date', 'Time'],
axis=1).dropna().reset_index(drop=True)
data['hour_sin'] = np.sin(2*np.pi*data['datetime'].dt.hour/24)
data['hour_cos'] = np.cos(2*np.pi*data['datetime'].dt.hour/24)
data['is_weekend'] = (data['datetime'].dt.dayofweek >=
5).astype(int)

# Features & Normalisation
cols = ['Global_active_power', 'Global_reactive_power', 'Voltage',
        'Global_intensity', 'Sub_metering_1', 'Sub_metering_2',
        'Sub_metering_3', 'hour_sin', 'hour_cos', 'is_weekend']
ds = MinMaxScaler().fit_transform(data[cols].values)[-100000:]
tgt = MinMaxScaler().fit(data[['Global_active_power']])

# Sequences
def make_seq(d, L=60):
    X=[d[i:i+L] for i in range(len(d)-L)]
    y=[d[i+L,0] for i in range(len(d)-L)]
    return np.array(X), np.array(y)
X, y = make_seq(ds)
sp = int(len(X)*.8)
to_t = lambda a: torch.from_numpy(a).float()
Xtr,ytr = to_t(X[:sp]), to_t(y[:sp]).unsqueeze(-1)
Xte,yte = to_t(X[sp:]), to_t(y[sp:]).unsqueeze(-1)

# Modle
class StackedLSTM(nn.Module):
```

```

def __init__(self, inp, h=128, nl=3, dr=0.3):
    super().__init__()
    self.lstm=nn.LSTM(inp, h, nl, batch_first=True, dropout=dr)
    self.fc=nn.Linear(h, 1)
    def forward(self, x): return self.fc(self.lstm(x)[0][:, -1, :])

model=StackedLSTM(10); crit=nn.MSELoss()
opt=torch.optim.Adam(model.parameters(), lr=1e-3)

# Entranement
for ep in range(10):
    model.train()
    for i in range(0, len(Xtr), 32):
        opt.zero_grad(); l=crit(model(Xtr[i:i+32]), ytr[i:i+32])
        l.backward(); opt.step()

# valuation
model.eval()
with torch.no_grad(): yp=model(Xte)
yr=tgt.inverse_transform(yte.numpy().reshape(-1, 1))
yh=tgt.inverse_transform(yp.numpy().reshape(-1, 1))
print(f'MAE : {mean_absolute_error(yr, yh):.4f} kW') # 0.1912
kW
print(f'RMSE : {mean_squared_error(yr, yh)**.5:.4f} kW') # 0.2898
kW

```